

Avaliação Prática – CRUD com Autenticação JWT

Contexto

Você é responsável por desenvolver um módulo de gerenciamento de **Sala de Reuniões**. O sistema deve permitir que um usuário autenticado realize operações de cadastro, consulta, edição e exclusão (CRUD) de salas de reunião.

Requisitos Técnicos

- **Backend:** C#, Minimal API, JWT, Entity Framework
- **Frontend:** React com chamadas à API
- **Autenticação:** Login com geração de token JWT
- **Tabela única:** SalasReuniao (campos abaixo)

Tabela: SalasReuniao

Campo	Tipo
Id	int (PK, identity)
Nome	string (máx 100)
Capacidade	int
PossuiProjektor	bool

O que você deve entregar

1. **Backend** – código completo da API (você mesmo escreve)
2. **Frontend** – utilize o código HTML+React fornecido abaixo, completando os trechos indicados por // COMPLETAR

Código HTML (frontend)

Atenção: Este código está pre-preparado no arquivo index.html

Copie o código abaixo e salve como index.html. Ele já contém a estrutura básica, estilos simples e o esqueleto do React com comentários.

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gestão de Salas de Reunião</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    .container { max-width: 800px; margin: auto; }
    form { border: 1px solid #ccc; padding: 15px; margin-bottom: 20px;
border-radius: 5px; }
    input, button { margin: 5px 0; padding: 8px; }
    table { width: 100%; border-collapse: collapse; margin-top: 20px; }
    th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
    .error { color: red; }
    .success { color: green; }
  </style>
  <script crossorigin
src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script crossorigin src="https://unpkg.com/react-dom@18/umd/react-
dom.development.js"></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head>
<body>
<div class="container" id="root"></div>

<script type="text/babel">
// ----- API BASE (COMPLETAR) -----
const API_URL = "http://localhost:5000"; // COMPLETAR com a porta da sua
API

// ----- COMPONENTE LOGIN -----
function Login({ onLogin }) {
  const [email, setEmail] = React.useState("");
  const [senha, setSenha] = React.useState("");
  const [erro, setErro] = React.useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();
    setErro("");
```

```

    try {
      // COMPLETAR: chamada POST para /login com email e senha
      // O backend deve retornar { token: "..." }
      // Salvar token no localStorage e chamar onLogin(token)
    } catch (err) {
      setErro("Falha no login. Verifique suas credenciais.");
    }
  };

  return (
    <div>
      <h2>Login</h2>
      <form onSubmit={handleSubmit}>
        <div>Email: <input type="email" value={email} onChange={e =>
setEmail(e.target.value)} required /></div>
        <div>Senha: <input type="password" value={senha} onChange={e
=> setSenha(e.target.value)} required /></div>
        <button type="submit">Entrar</button>
        {erro} && <div className="error">{erro}</div>
      </form>
    </div>
  );
}

// ----- COMPONENTE CRUD SALAS -----
function CrudSalas({ token }) {
  const [salas, setSalas] = React.useState([]);
  const [nome, setNome] = React.useState("");
  const [capacidade, setCapacidade] = React.useState("");
  const [possuiProjektor, setPossuiProjektor] = React.useState(false);
  const [editandold, setEditandold] = React.useState(null);
  const [mensagem, setMensagem] = React.useState("");

  // Carregar lista de salas ao montar o componente
  React.useEffect(() => {
    carregarSalas();
  }, []);

  async function carregarSalas() {
    try {
      // COMPLETAR: GET /salas com header Authorization: Bearer
{token}
      // Atualizar state 'salas'
    } catch (err) {
      setMensagem("Erro ao carregar salas");
    }
  }

  async function salvarSala(e) {
    e.preventDefault();

```

```

const sala = { nome, capacidade: parseInt(capacidade), possuiProjeto: };

try {
  if (editando === null) {
    // COMPLETAR: POST /salas com token
  } else {
    // COMPLETAR: PUT /salas/{id} com token
  }
  setMensagem("Operação realizada com sucesso!");
  limparForm();
  carregarSalas();
} catch (err) {
  setMensagem("Erro ao salvar sala");
}
}

async function excluirSala(id) {
  if (!confirm("Tem certeza?")) return;
  try {
    // COMPLETAR: DELETE /salas/{id} com token
    setMensagem("Sala excluída!");
    carregarSalas();
  } catch (err) {
    setMensagem("Erro ao excluir");
  }
}

function editarSala(sala) {
  setEditando(sala.id);
  setNome(sala.nome);
  setCapacidade(sala.capacidade);
  setPossuiProjeto(sala.possuiProjeto);
}

function limparForm() {
  setEditando(null);
  setNome("");
  setCapacidade("");
  setPossuiProjeto(false);
}

return (
  <div>
    <h2>Gerenciar Salas de Reunião</h2>
    {mensagem && <div className="success">{mensagem}</div>}
    <form onSubmit={salvarSala}>
      <div>Nome: <input value={nome} onChange={e =>
setNome(e.target.value)} required /></div>
      <div>Capacidade: <input type="number" min="1"
value={capacidade} onChange={e => setCapacidade(e.target.value)} required

```

```

/></div>
    <div>
      <label>Possui Projetor:</label>
      <input type="checkbox" checked={possuiProjetor} onChange={e
=> setPossuiProjetor(e.target.checked)} />
    </div>
    <button type="submit">{editandold ? "Atualizar" :
"Adicionar"}</button>
    <button type="button" onClick={limparForm}>Cancelar</button>
  </form>

  <table>
    <thead>

<tr><th>ID</th><th>Nome</th><th>Capacidade</th><th>Projetor</th><th>Ações</th></tr>
    </thead>
    <tbody>
      {salas.map(sala => (
        <tr key={sala.id}>
          <td>{sala.id}</td>
          <td>{sala.nome}</td>
          <td>{sala.capacidade}</td>
          <td>{sala.possuiProjetor ? "Sim" : "Não"}</td>
          <td>
            <button onClick={() => editarSala(sala)}>Editar</button>
            <button onClick={() =>
excluirSala(sala.id)}>Excluir</button>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
);
}

// ----- COMPONENTE PRINCIPAL -----
function App() {
  const [token, setToken] = React.useState(localStorage.getItem("token"));

  const handleLogin = (novoToken) => {
    localStorage.setItem("token", novoToken);
    setToken(novoToken);
  };

  const handleLogout = () => {
    localStorage.removeItem("token");
    setToken(null);
  };

```

```

    if (!token) {
      return <Login onLogin={handleLogin} />;
    }

    return (
      <div>
        <button onClick={handleLogout} style={{float: "right"}}>Sair</button>
        <CrudSalas token={token} />
      </div>
    );
  }

ReactDOM.createRoot(document.getElementById("root")).render(<App />);
</script>
</body>
</html>

```

Instruções para execução

1. Backend – Crie uma Minimal API com:

- Rota POST /login que recebe { "email": "admin@email.com", "senha": "123456" } e retorna um token JWT (qualquer usuário fixo, sem banco de dados de usuários, para simplificar).
- Rota POST /salas (criar sala)
- Rota GET /salas (listar todas)
- Rota PUT /salas/{id} (atualizar)
- Rota DELETE /salas/{id} (excluir)
- Use Entity Framework com SQLite ou SQL Server para a tabela SalasReuniao.
- Proteja as rotas de CRUD com autenticação JWT.

2. **Frontend** – No código HTML acima, complete todas as ocorrências de // COMPLETAR com as chamadas fetch apropriadas, enviando o token no header Authorization: Bearer {token} e tratando os responses.

3. **Entrega** – Envie o código do backend (arquivos `.cs`) e o arquivo `index.html` funcionando.

Critérios de avaliação

Critério	Pontuação
Backend: login com JWT funcionando	1,0
Backend: CRUD completo da tabela SalasReuniao	1,5
Frontend: chamadas para API completas e corretas	1,5
Frontend: exibição dos dados e ações (CRUD)	0,75
Organização e simplicidade do código	0,25
Total	5,0

Dicas

- Teste primeiro as rotas da API com Postman/Insomnia antes de integrar com o frontend.
- No backend, você pode usar um único usuário fixo para simplificar (ex: email: "teste@teste.com", senha: "123").
- O frontend já está configurado para usar `localStorage` para guardar o token.
- Mantenham o código simples – o objetivo é verificar o funcionamento do CRUD e da autenticação.

Bom trabalho!